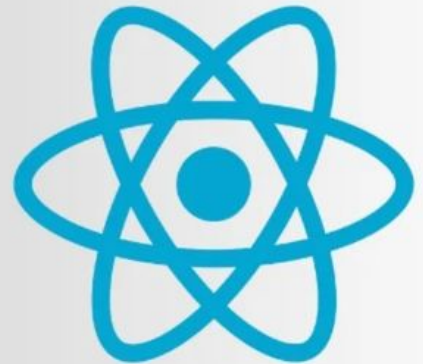


REACT ALL **15** HOOKS EXPLAINED



@frontend_in_depth

useState Hook

This hook allows you to store and change information within a component. It returns a pair of the current state, a function that lets you update it and the value of the state



```
const [stateVariable, setStateFunction] = useState(initialvalue);
```

UseRef Hook

This hook lets you reference a value in your component. It doesn't cause the component to re-render when it changes and it remembers its value even after the component re-renders.

```
const myRef = useRef(initialValue);
```

UseEffect Hook

This hook lets you perform side effects like data fetching, updating the DOM, or subscribing to events.

```
useEffect(() => {  
  // Side effect code here  
  return () => {  
    // Cleanup code (optional)  
  };  
}, [dependencies]) ;
```

Use context Hook

React Context is a way to manage state globally, It helps you pass data around components without having to manually pass it through props.

```
import React, { createContext, useContext } from 'react';

const MyContext = createContext ();

const ParentComponent = () => (
  <MyContext.Provider value="Hello from context!">
    <childComponent />
  </MyContext.Provider>
);

const ChildComponent = () => {
  const contextValue = useContext(MyContext);
  return <p>{contextValue}</p>;
};
```

UseMemo hook

This hook remembers the result of a function and only re-calculates if the inputs change.

```
const cachedValue = useMemo(calculateValue, dependencies)
```

UseRefuser Hook

This hook is used for managing complex state logic that involves multiple sub-values or when the next state depends on the previous one

```
const [state, dispatch] = UseRefuser(reducerFunction, initialArg, initFunction) ;
```


UseCallback Hook

This hook is used to memoize (remember) functions, so you don't have to recreate it every time your component re-renders, unless something it depends on changes.

```
const memoizedCallback = useCallback(callbackFunction, dependencies);
```


UseImperativeHandle

This hook allows you to control what is returned by a ref prop, specifically the ref handle.



```
useImperativeHandle(ref, createHandle, dependencies?).
```

UseDebugvalue hook

This hook allows developers to add a label or description for custom hooks, it helps developers understand the purpose and usage of custom hooks when inspecting components in browser developer tools.



```
useDebugValue(value, format);
```

UseDeferredvalue hook

This hook allows you to postpone a value change that might cause a component to lag during rendering.



```
const deferredValue = useDeferredValue(value, options);
```

UseId hook

This is a custom hook that generates unique IDs within a React component.



```
const id = useId();
```

UseInsertionEffect

`useInsertionEffect` is a custom hook that allows you to insert the styles before any layout effects fire

```
useInsertionEffect(setup, dependencies?)
```

UseLayoutEffect hook

This hook is similar to `useEffect`, but it runs after all DOM mutations are complete. This makes it useful for reading layout from the DOM and synchronizing state.



```
useLayoutEffect(() => {  
  // Do some thing and either return undefined or a cleanup function  
  return () => {  
    // Do some cleanup here  
  };  
}, [dependencies]);
```

UseSyncExternalStore

`useSyncExternalStore` is a React Hook that lets you subscribe to an external store.

```
const snapshot = useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)
```


UseTransition Hook

useTransition is a React Hook that lets you update the state without blocking the UI.

```
const [isPending, startTransition] = useTransition()
```



Follow for more



Halim Shams